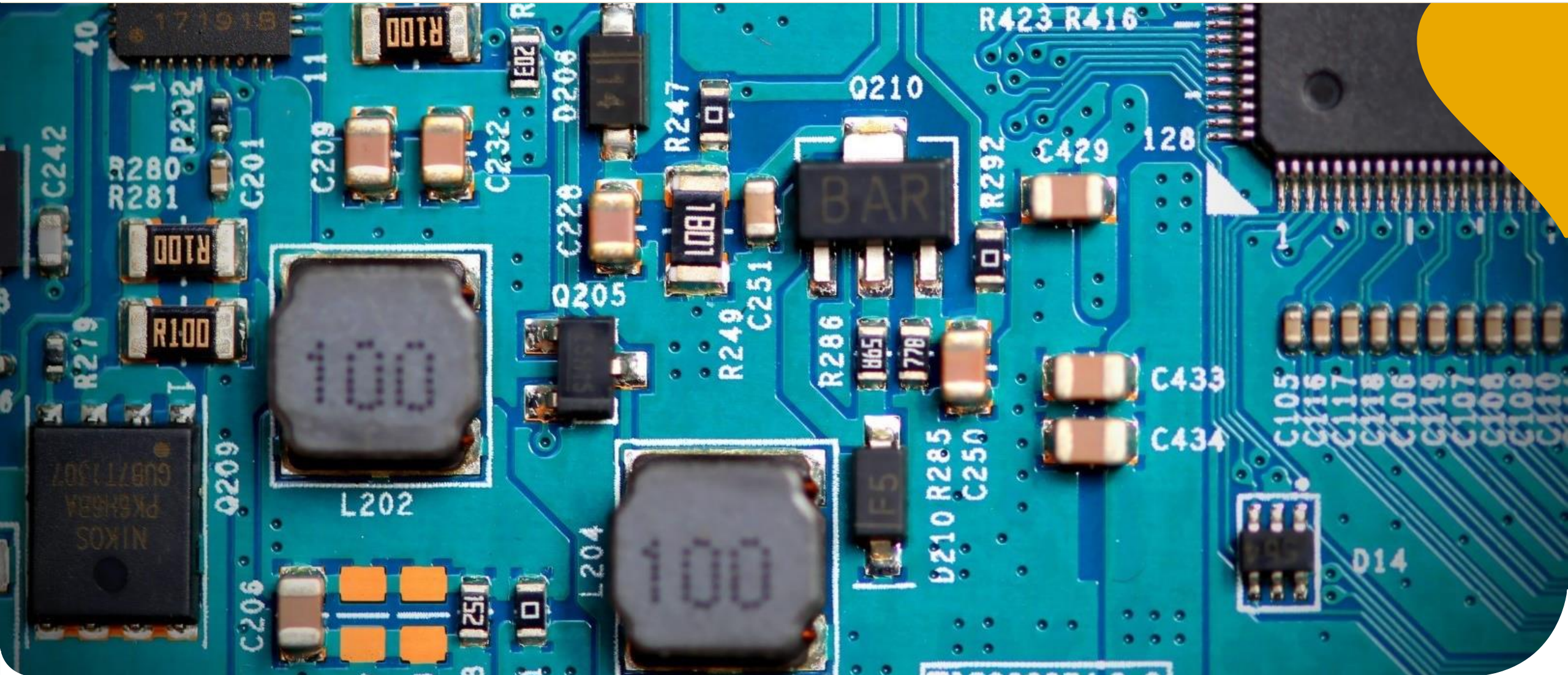


# Design(E)-314

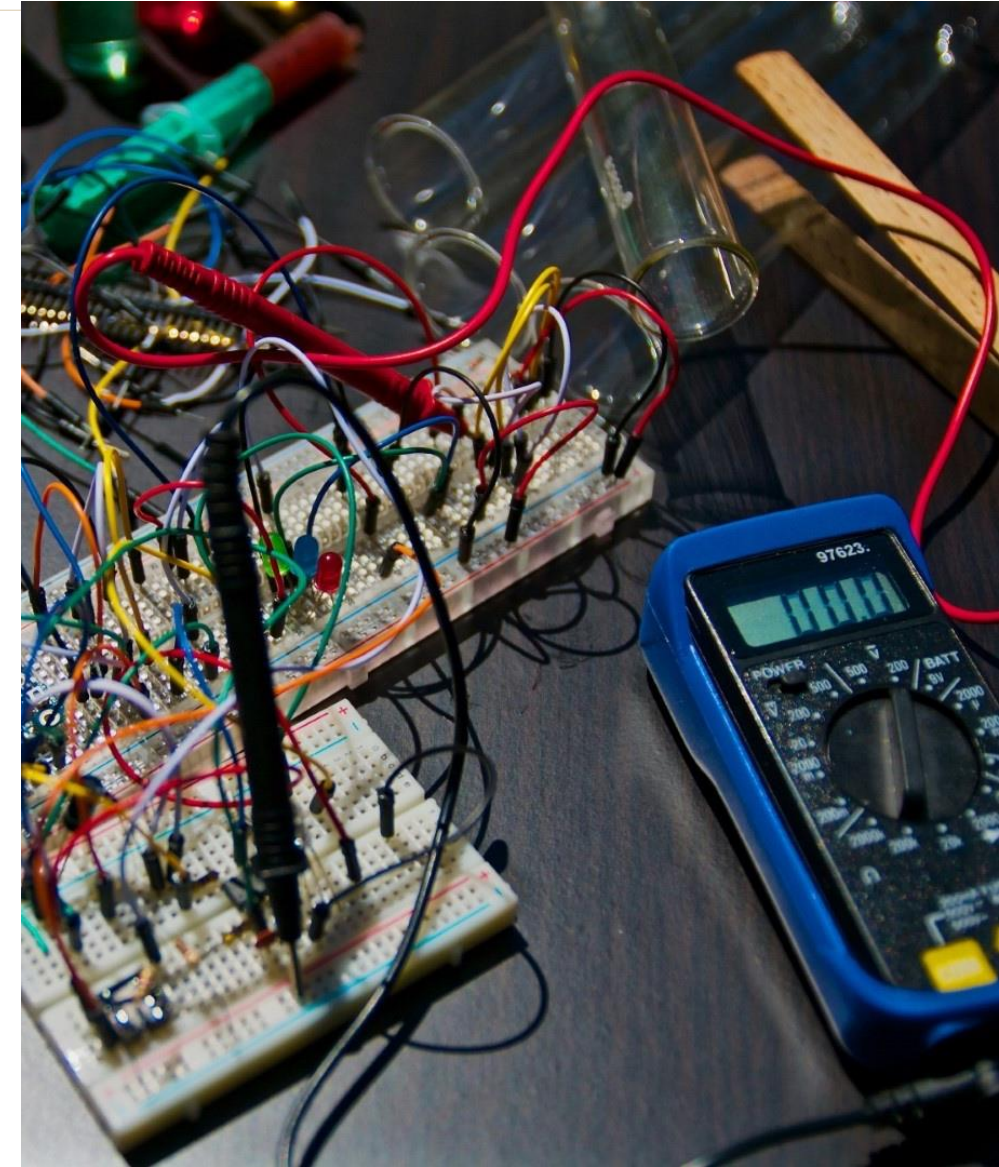
## Lecture 2



# Overview

## We are in the Demo 1 cycle

- Admin
- PDD – updates
- Demo 1 requirements
- Practical 1
- Practical 2
  - GIT, GPIO Buttons
  - Debouncing
  - Software Interrupts
  - Temperature sensor
  - UART



## Exemptions?

- No exemptions

## Lectures & Tests (3x Tests written during class lecture time slots)

- Tuesdays: 08:10 – 09:00 M&M
- Tuesdays: 09:10 – 10:00 E&E (all focus areas) and Comp. Science

## Practicals

- Thursdays: 09:00 – 12:00 M&M and Comp. Science
- Fridays : 10:00 – 13:00 E&E (all focus areas)

# PDD - Updates

- Version 0.2 uploaded

## Version History

| Version | Date        | Changes                                       |
|---------|-------------|-----------------------------------------------|
| 0.1     | 9 Feb 2026  | - Initial project specification and guide     |
| 0.2     | 11 Feb 2026 | - Added Demo 1 information in Tables 9 and 10 |

- Note: 3.3V regulator
  - There are 2x different 3.3V regulators used this year
  - LF33 & LD1117V3 (make sure you look at the right data sheet)  
↳ mine works so it's fine 😊

# Demo 1 requirements

- PDD Tables 9 & 10

| Demo | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Relevant SS's                                                                                                            |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| 1    | On power-up, the TS will expect your student number via UART using the prescribed message format, no sooner than 100ms after power-up and no later than 500ms after power-up. The TS will simulate button pushes with and without bounce for push-buttons S1-S5. The TS will override the tactile push-buttons (via TIC connector P13 pins 7, 8, 9, 10, 11) while taking photographs (of the status-indicating LEDs) to check for correct system operation. The maximum response time after button pushes is 500 ms. The TS will monitor and test the output of the digital temperature sensor (via UART). The TS will visually monitor the power status LED (D6) and status-indicating LEDs (D2-D5). The TS will monitor and test the 3.3V and 5V lines. GIT repos will be checked. | SS1, SS2 (student number), SS6 (Temperature), SS7 (LEDs), SS8 (tactile push-buttons), SS12 (5V, 3.3V, UART-RX, UART-TX). |

Table 10: Demo descriptions

# Practical 1 Feedback

## 5V Regulator

\* We need to find out why the application circuits are constructed in that manner

- Design provided in PCB schematic file
- What is the point of the capacitors?

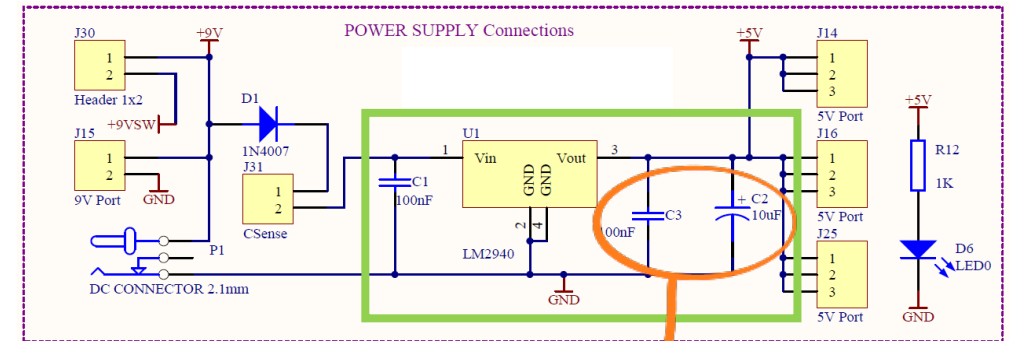


- What is the purpose of the D1 diode?  
↳ Reverse polarity protection

## 3.3V Regulator

- Why should there be cap. on the input and output?
- Should I follow the datasheet design notes?
- Can I use a bigger Cap. on the output?

\* The capacitors stabilise the signals moving through the board



Serves as an input to 3.3V regulator

### Pro-Tip:

Go read up on Bypass capacitors.

Confirm a stable, “ripple free” output on the regulators.



# Practical 1

## Feedback



Stellenbosch  
UNIVERSITY  
IYUNIVESITHI  
UNIVERSITEIT

forward together  
sonke siya phambili  
saam vorentoe



### Pro-Tip:

Lower  $V_{in}$  = better efficiency. Why?

## 9V supply to 5V Regulator & 3.3V Regulator

- Yes, you can supply the 3.3V with 9V as well, or even 12V.
- Supply 3.3V regulator with 9V? Or should the 5V output supply the 3.3V?
  - ↳ Question of safety
  - ↳ If the 3.3V fails, the STM will receive the full 9V ∴ **EXPLOSION**

## Heatsink



↳ If the 5V fails, the 3.3V can still handle the 9V and protect the STM

- How do I know if I should use a heat sink? Well...ask yourself...
- What is my expected power dissipation?
  - ↳ Measure it continuously, because it might be necessary to further down the line
- Estimate temperature rise with thermal resistance ( $^{\circ}\text{C}/\text{W}$ )



### Pro-Tip:

Look for the "Maximum Junction Temperature" and "Thermal Resistance"

# Practical 2

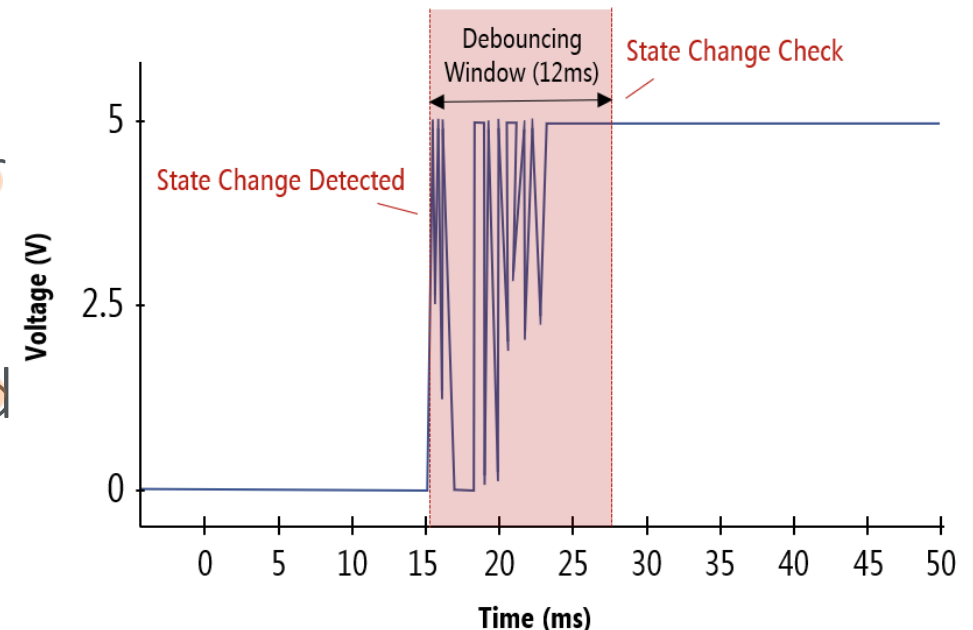
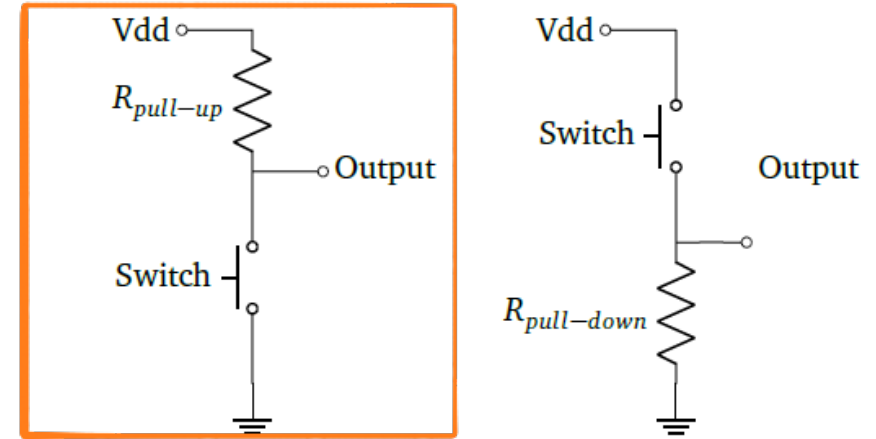
## GPIO buttons & Interrupts



Stellenbosch  
UNIVERSITY  
IYUNIVESITHI  
UNIVERSITEIT

forward together  
sonke siya phambili  
saam vorentoe

- Your GPIO buttons should be configured as External Interrupts in the IDE
- Active High vs. Active Low (what do you need?)
- Pull-up or Pull-down resistor:
  - You can use an external resistor
  - You can use GPIO Pull-up/Pull-down internal resistor
- External interrupt GPIO mode can be configured as either Rising, Falling or Rising/Falling edge detection



# Practical 2

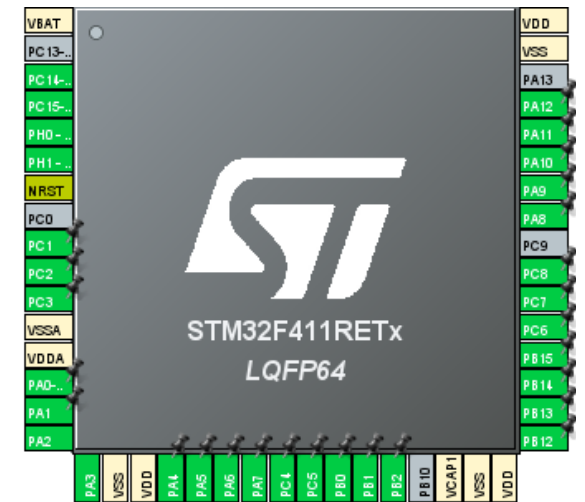
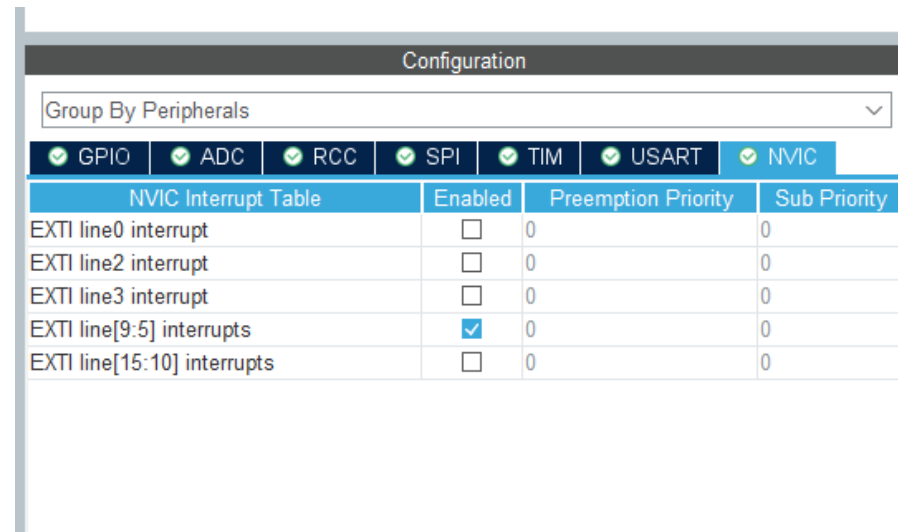
## GPIO buttons & Interrupts



Stellenbosch  
UNIVERSITY  
IYUNIVESITHI  
UNIVERSITEIT

forward together  
sonke siya phambili  
saam vorentoe

- External interrupts needs to be activated in the IDE



- Be careful: Hardware Architecture Limitation

# Practical 2

## GPIO buttons & Interrupts

inductor  $\rightarrow$   $i$  can't change instantaneously  
capacitor  $\rightarrow$   $v$  can't change instantaneously

- Be careful: Interrupt Hardware Architecture Limitation
- Multiple ports feed into a multiplexer that routes to each EXTI line.
- This means the multiplexer EXTI line can only select one GPIO port per pin number, at any given time
- You can only select one GPIO port per pin number at any given time....which means...you can only use pins with different numbers simultaneously as interrupts

**\*NB**

- Example: PA4 and PB4 cannot both be used as EXTI
- Example: PA4 and PB5 can be used as EXTI

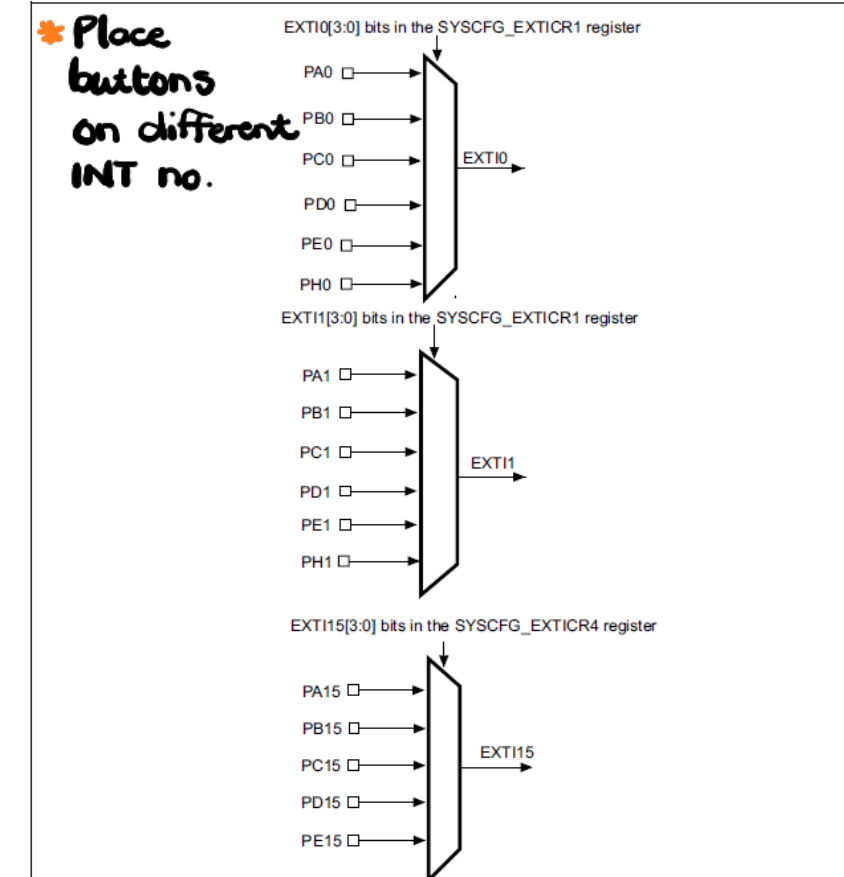
### Interrupts and events

RM0383

#### 10.2.5 External interrupt/event line mapping

Up to 81 GPIOs (STM32F411xC/E) are connected to the 16 external interrupt/event lines in the following manner:

Figure 30. External interrupt/event GPIO mapping



MS31425V1

# Practical 2

## GPIO buttons & Interrupts

### Software application

- Interrupt >> Callback Function >> Execute code
- The HAL library provides a high-level access to STM32 peripherals like the EXTI.
- Go get the callback function in the STM32F4xx\_hal\_gpio.c file
- Copy and paste the code to main.c (delete \_\_weak pretext before void)

```
/* USER CODE BEGIN 4 */  
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)  
{  
    if(GPIO_Pin == GPIO_PIN_13) {  
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);  
    } else {  
        __NOP();  
    }  
}  
/* USER CODE END 4 */
```

### Pro-Tip:

STM has a pretty nice website... <https://wiki.st.com/stm32mcu/wiki/Microcontroller>  
search for “Getting started with EXTI”



# Practical 2

## GPIO buttons & Interrupts

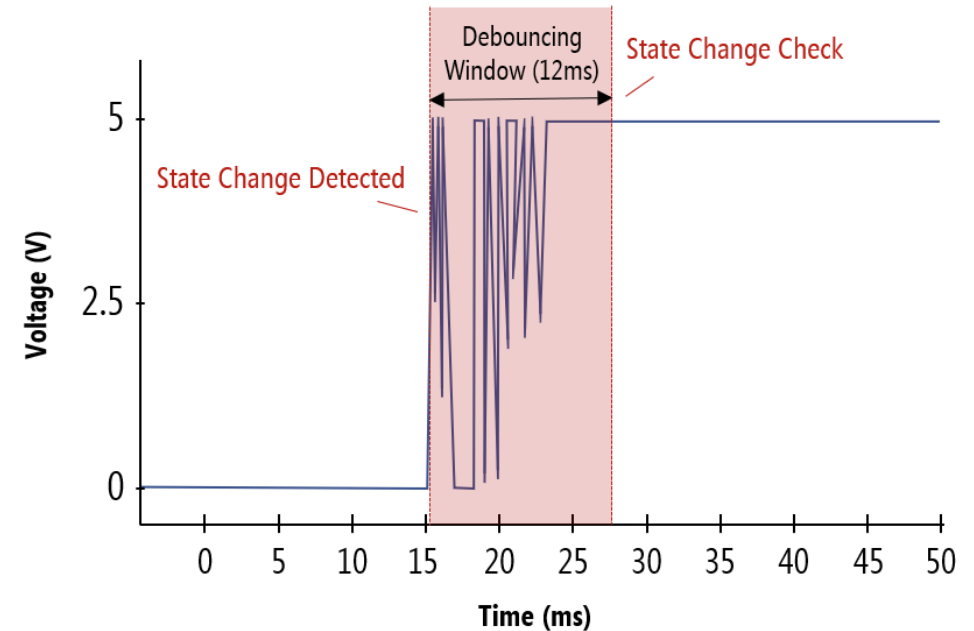


Stellenbosch  
UNIVERSITY  
IYUNIVESITHI  
UNIVERSITEIT

forward together  
sonke siya phambili  
saam vorentoe

### Software application

- For the Button, your EXTI might be executed several times due to mechanical bounce.
- How can you handle button debouncing in software? (vs. Hardware)
  - ↳ You can put a delay  
OR
  - ↳ You can record the time passed from when the button was pressed
    - If enough time has passed then, you can take a reading



### Pro-Tip:

To ensure that your button functions the way it should, identify the GPIO that triggered the interrupt AND identify the pin state (high or low) to avoid another accidental trigger upon button release

# Practical 2

## Software (interrupts)



### Software/Code for Interrupts:

- Typically, only set a trigger flag in the interrupt callback function
- Avoid large segments of code in the callback function (this keeps the main loop from continuing)
- Set a flag >> main loop checks for flag condition >> main loop executes function

As a global variable → volatile

↳ Preventative measure to stop the compiler from removing variables

### General Software design

- Good Software Design follows Modular design
- Separating the functionality of a program into independent, interchangeable modules

**Volatile** type  
for variables...

Interrupts  
cannot change  
variable unless  
volatile



# Practical 2

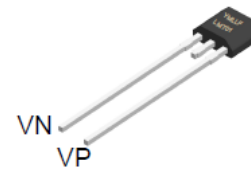
## Temperature sensor

### LMT01 Digital Temp Sensor

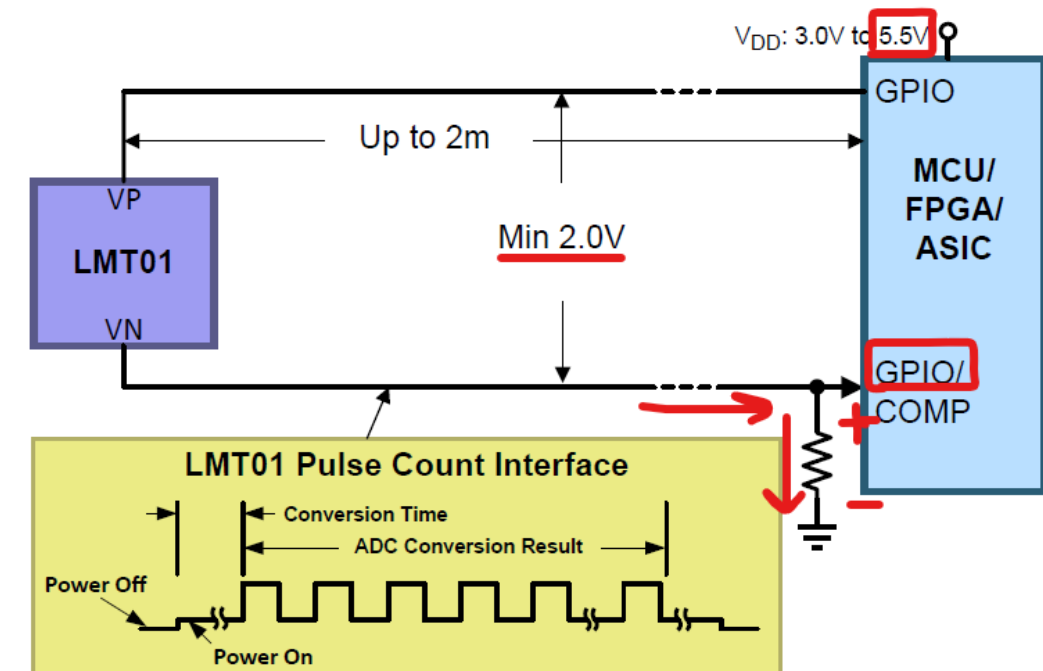
- LMT01 delivers a pulse according to the temperature value.
- Pulse is delivered by means of an output current.
  - Can STM measure current?
    - No, we need to use internal resistor to convert voltage reading to current reading
- Minimum 2V required across VP and VN



LPG Package  
2-Pin TO-92  
Top View



### 2-Pin IC Temperature Sensor



# Practical 2

## Temperature sensor

### Timing

- 50ms between start and end of data transmission
- 104ms between start of data transmission until start of next data transmission

### Conversion

- 1x Pulse Count = 0.0625 °C

#### LMT01

SNIS189D – JUNE 2015 – REVISED JUNE 2018

www.ti.com

#### Feature Description (continued)

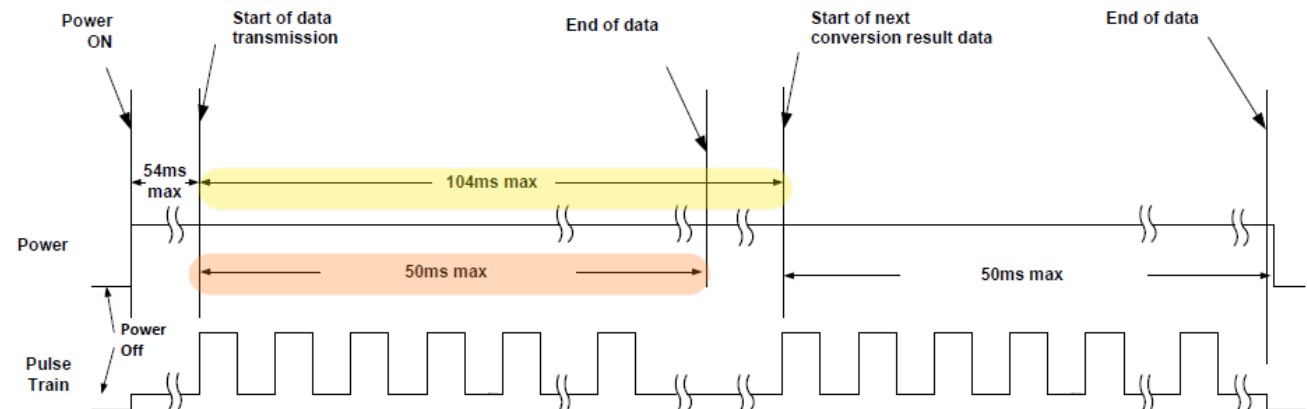


Figure 22. Temperature to Digital Pulse Train Timing Cycle

The LMT01 can be powered down at any time to conserve system power. Take care to ensure that a minimum power-down wait time of 50 ms is used before the device is turned on again.

\*NB

$$\text{Temp} = \left( \frac{\text{PC}}{4096} \times 256^{\circ}\text{C} \right) - 50^{\circ}\text{C}$$

where

- PC is the Pulse Count
- Temp is the temperature reading

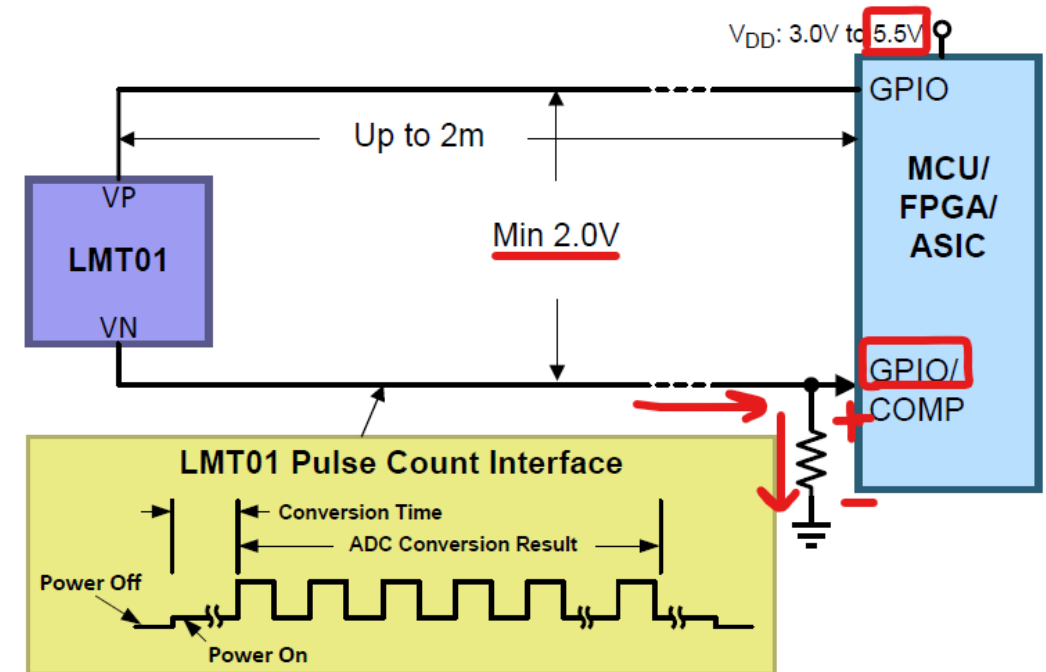
# Practical 2

## Temperature sensor

### Note

- Our GPIO pins cannot read current values?
- Minimum 2V required across VP and VN
- How will we read “low” and “high” pulse?
- $V = IR$
- For the STM32 MCU, what is the minimum input high (VIH) and maximum input low voltage (VIL)?  $I = \frac{V_{IL} - V_{IH}}{R}$

### 2-Pin IC Temperature Sensor



**Pro-Tip:**  
Look for VIH and VIL in  
STM32 datasheet

# Practical 2

## Temperature sensor

### Software & Timing

- Implementation: You can use either EXTI or TIMER (external clock mode) to count pulses...
- If the EXTI is used for each pulse count, will this not overload the CPU?



#### Pro-Tip:

For digital sampling, you take multiple (5-10) measurements, throw away the lowest and highest values, and average out the remaining to get a more accurate mean and avoid anomalies.

\* $f = 88\text{kHz}$   
↳ Is the microcontroller fast enough to take those readings

## 2x Modes of UART receive

- Receive UART Data in **Blocking Mode**
- CPU is blocked and it waits for the required number of bytes to arrive or the timeout, whatever happens first.

```
while (1)
{
    HAL_UART_Receive(&huart2, RxData, 5, 1000);

    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    HAL_Delay(1000);
}
```

- Will the LED blink every 1sec or every **2 sec**?

# UART



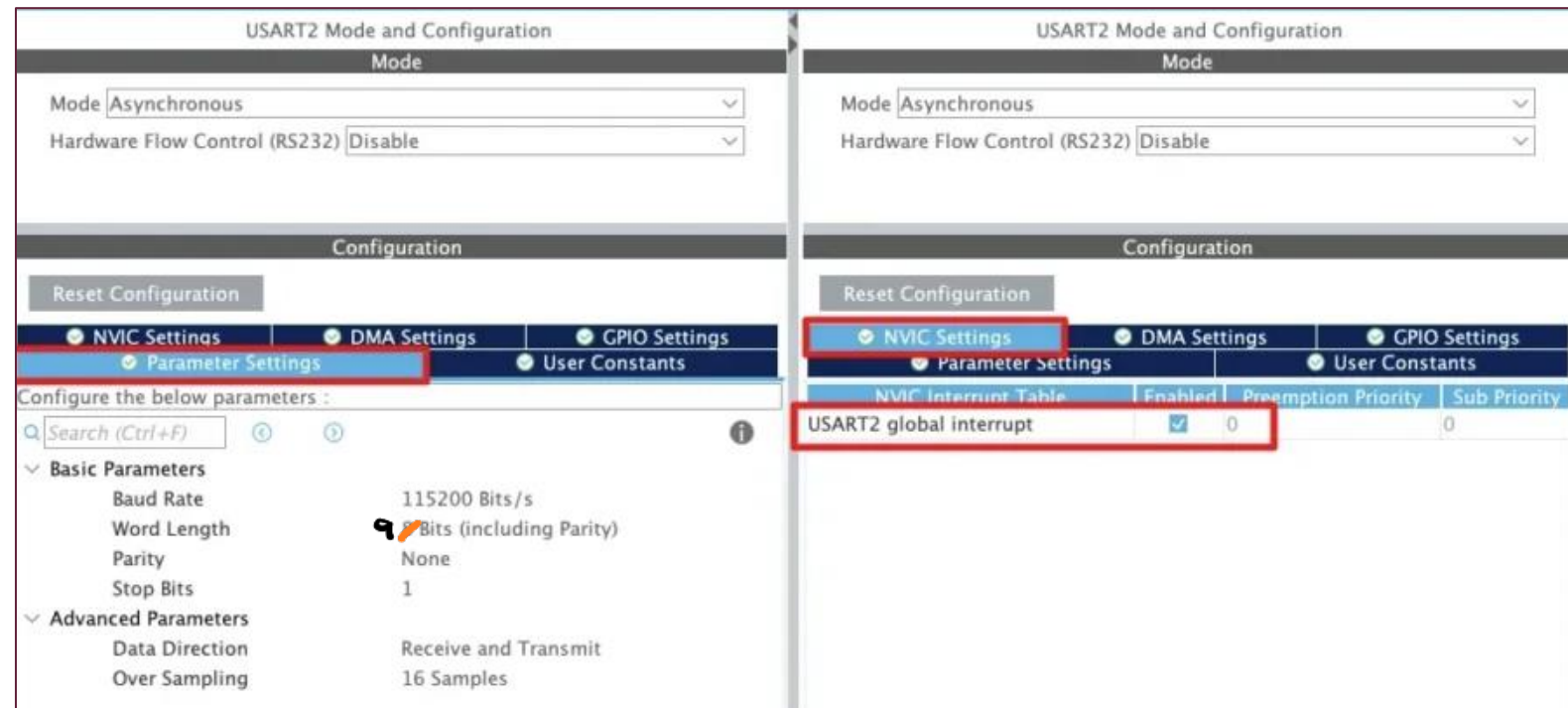
## 2x Modes of UART receive

- Receive UART Data in **Interrupt Mode**

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    // do something with the data
}
```

- Receive the data in the background.
- CPU can process other tasks as usual and once the required number of bytes have been received, the interrupt will trigger.

```
int main ()
{
    .....
    HAL_UART_Receive_IT(&huart2, RxData, 5);
    while (1)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        HAL_Delay(1000);
    }
}
```



## RX: Fixed vs. Unknown Data Length

- How can we receive data of an unknown length?
- Basically we will receive 1 byte at a time and store it in a buffer.
- Monitor the incoming byte for the **terminating character** ('\n').....then process the data

### Pro-Tip:

You cannot receive anything on the RX line, unless you have actively instructed the STM32 to listen on the RX line.

Initialise before main loop:

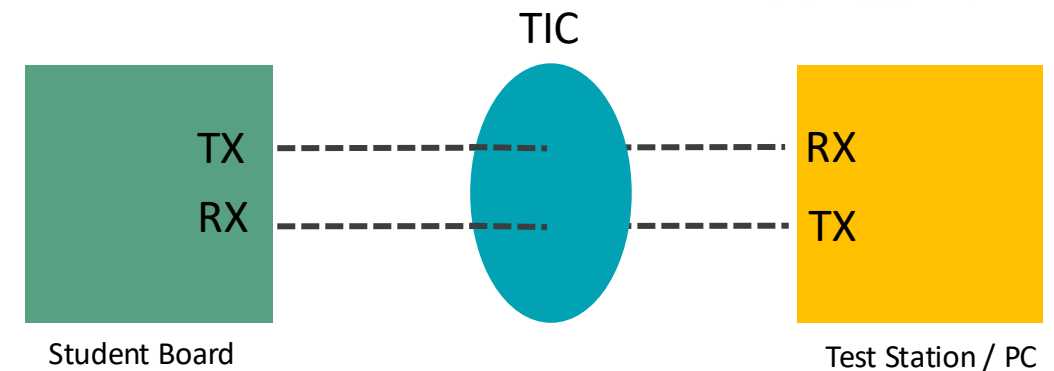
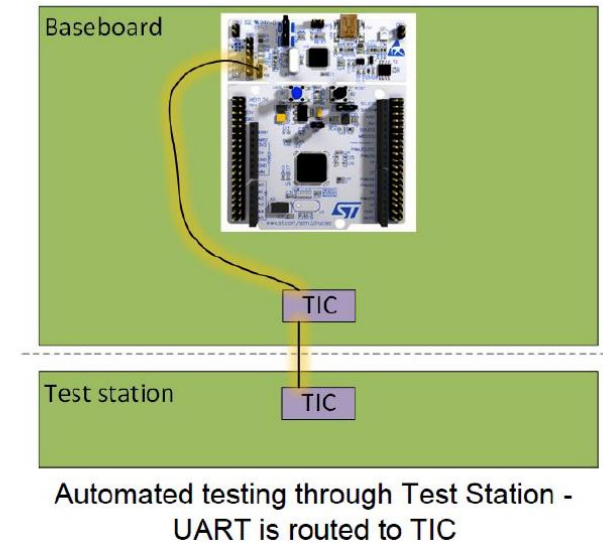
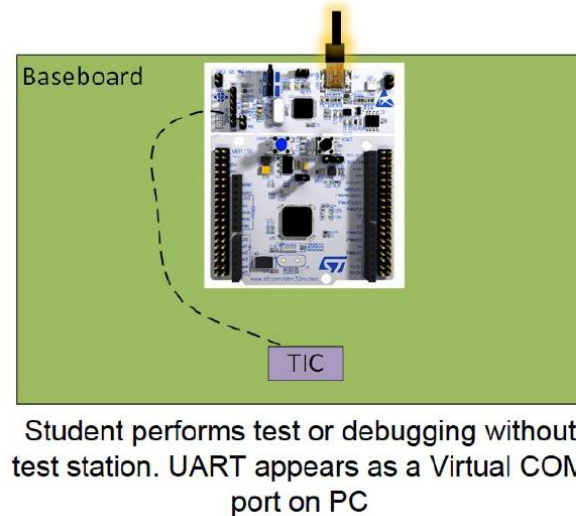
```
HAL_UART_Receive_IT(&huart2, temp, 1);
```

Initialise again, after processing incoming data



## Test Interface Connector (TIC)

- Be careful about your connections
- Table “*Test-interface Connector Pin definitions*”

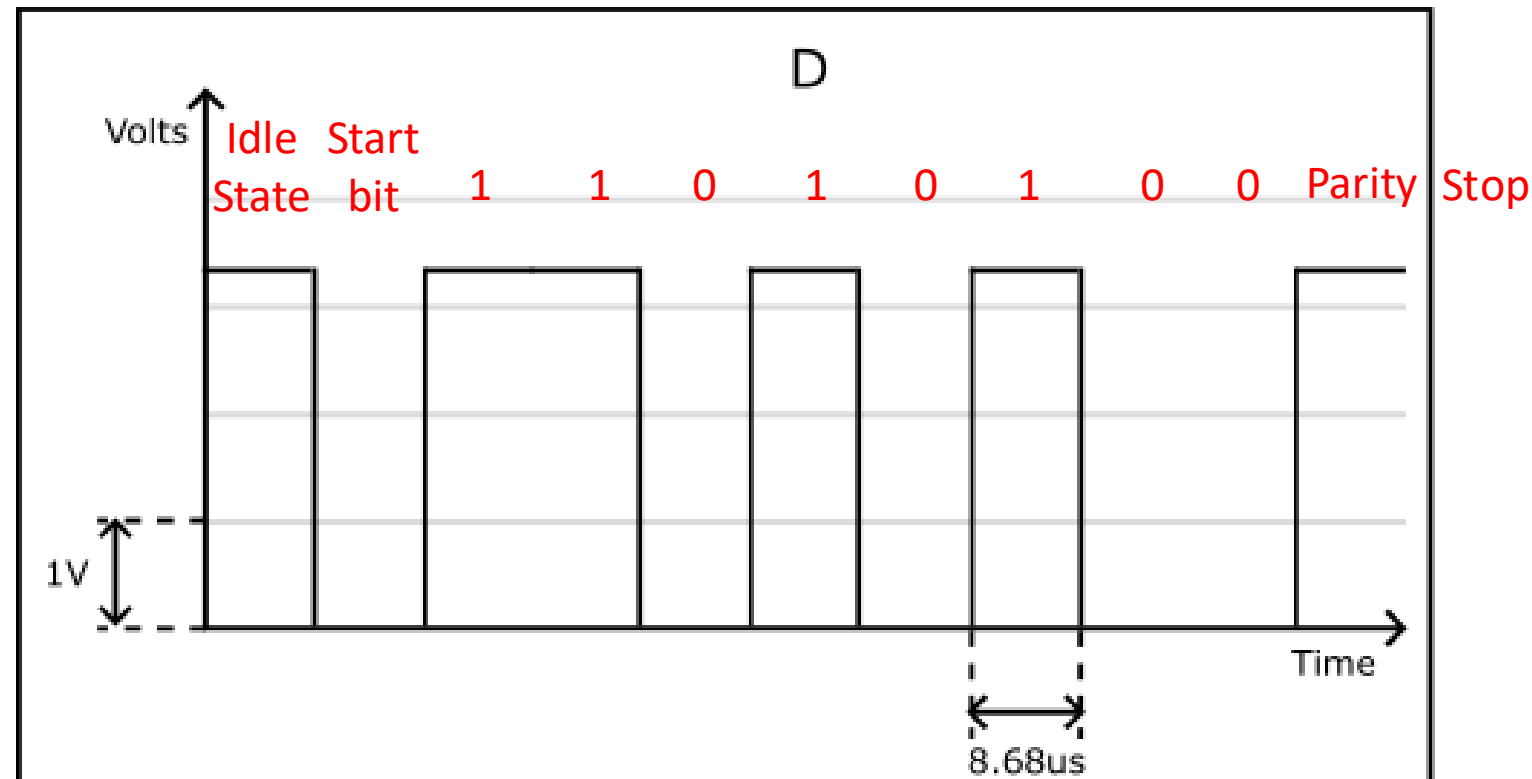


# UART

## Measurement exercise

Confirm in your measurements!

- Start
- Stop
- Voltage
- Baud rate
- Parity
- Bit order



# Questions?

